Name of the Student	D.Chinna Moulali
Reg. No	192425278
GitHub Link	https://github.com/kiduuuu/MLA0405-Deep-learning/tree/main

SIMATS ENGINEERING

CO5 - ASSESSMENT TOOL 1

Design Desk

COURSE NAME: Deep Learning **COURSE CODE: MLA0405**
SLOT : D **Faculty : Dr.Arul Raj A.M**

COURSE OUTCOME COVERED

CO5: Students will be able to analyze and explain advanced generative deep learning models including Autoencoders, Deep Belief Networks, Boltzmann Machines, Deep Boltzmann Machines, and Generative Adversarial Networks. They will be able to compare their architectures, explain their learning mechanisms, and apply suitable generative models to real-world problems. (BTL-3)

Course Outcome Weightage: 35% **Duration: 90 minutes**
Assessment Tool Weightage: 30% **Mark : 25**

Code of Conduct:
I (G.Harshitha / 192372384) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D.Chinna Moulali **Name of the student:** D.Chinna Moulali

Criteria	Subject Knowledge and Conceptual Understanding (2)	Technical Accuracy and Application of Concepts (3)	Problem-Solving and Analytical Thinking **(2)**	Communication Skills and Clarity of Explanation **(2)**	Confidence, Presentation and Professional Behaviour ***(1)***	**Total (10)**
Weightage	***20 %***	***30%***	***20%***	***20 %***	***10 %***	**100%**
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

Faculty Signature : **Total Marks:**

QUESTIONS: 25 **Total Marks:25**

1. Design an Autoencoder-Based Feature Learning System

Real-world problem: A manufacturing company needs to detect defective products using high-dimensional sensor data.

Requirements:

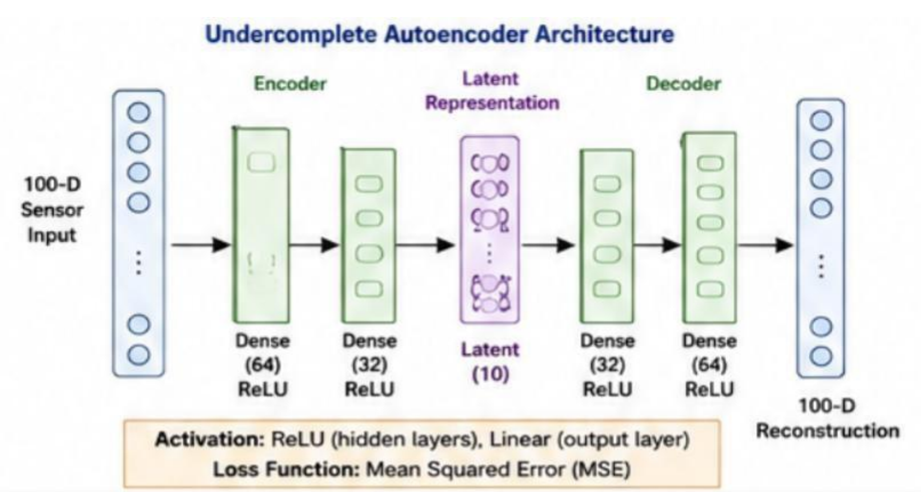
- **Input:** 100 sensor features such as temperature, pressure, vibration and voltage.
- **Output:** Reconstructed sensor data and a compact feature representation.
- **Constraint:** Reduce dimensionality while preserving important information.

Architecture:

100-D Sensor Input → Dense(64) → Dense(32) → Latent(10) → Dense(32) → Dense(64) → 100-D Reconstruction

Design:

- Encoder: 100 → 64 → 32 → 10 neurons.
- Latent representation: **10-dimensional feature vector**.
- Decoder: 10 → 32 → 64 → 100 neurons.
- Hidden activation: **ReLU**.
- Output activation: **Linear** for continuous sensor values.
- Reconstruction loss: **Mean Squared Error (MSE)**.



Working: The encoder compresses 100 sensor features into 10 important latent features, while the decoder reconstructs the original sensor data; the latent vector can then be used for defect classification or anomaly detection.

Evaluation: Reconstruction MSE, classification accuracy, precision, recall, F1-score and anomaly-detection performance.

Expected outcome: The system reduces data dimensionality by **90%** while learning important sensor patterns useful for identifying defective products.

2. Design a Robust Regularized Autoencoder

Real-world problem: A healthcare organization has noisy patient-record features and requires robust representations for classification.

Requirements:

- **Input:** Patient features such as laboratory values, vital signs and medical-record attributes.
- **Output:** Clean reconstruction and robust latent features.
- **Constraint:** Reduce noise and prevent overfitting.

Architecture:

Noisy Patient Data → Dense(128) → Dense(64) → Latent(16) → Dense(64) → Dense(128) → Reconstructed Data

Design:

- Encoder: 128 → 64 → 16 neurons.
- Decoder: 16 → 64 → 128 neurons.
- Hidden activation: **ReLU**.
- Output activation: **Linear/Sigmoid**, depending on data normalization.
- Regularization: **Dropout + L2 weight regularization**.
- Reconstruction loss: **MSE**.

Training objective:

Total Loss = Reconstruction Loss + $\lambda\|W\|^2$

Working: Dropout randomly removes neurons during training, while L2 regularization penalizes excessively large weights; together they reduce overfitting and encourage the model to learn useful general features.

Evaluation: Reconstruction error, classification accuracy, precision, recall, F1-score and validation loss.

Expected outcome: The model produces a more stable and noise-resistant representation for downstream patient classification.

3. Design a Deep Generative Model

Real-world problem: A recommendation/content-generation system needs to learn complex data distributions and generate new content.

Proposed model: Deep Belief Network (DBN)

Architecture:

Visible Layer → Hidden Layer 1 → Hidden Layer 2 → Hidden Layer 3

Example:

100 Input Features → 64 Hidden Units → 32 Hidden Units → 16 Hidden Units

Design:

- Visible layer: Represents input features.
- Hidden Layer 1: 64 neurons.
- Hidden Layer 2: 32 neurons.
- Hidden Layer 3: 16 neurons.
- Activation: **Sigmoid** for binary/Bernoulli features.
- Latent layer: 16-dimensional representation.

Learning process:

1. Train the first Restricted Boltzmann Machine (RBM).
2. Freeze its learned representation.
3. Train the next RBM using the previous hidden output.
4. Continue layer-by-layer.
5. Fine-tune the complete network for the desired application.
6. Generate new samples by sampling from the learned representation.

Why DBN? A DBN can learn hierarchical features and complex probability distributions from large datasets, making it suitable for recommendation and content-generation tasks.

Evaluation: Reconstruction error, recommendation accuracy, precision@K, recall@K and quality/diversity of generated samples.

4. Design a Stochastic and Contractive Representation Model

Real-world problem: An image-processing system needs representations that remain stable when images contain small changes or noise.

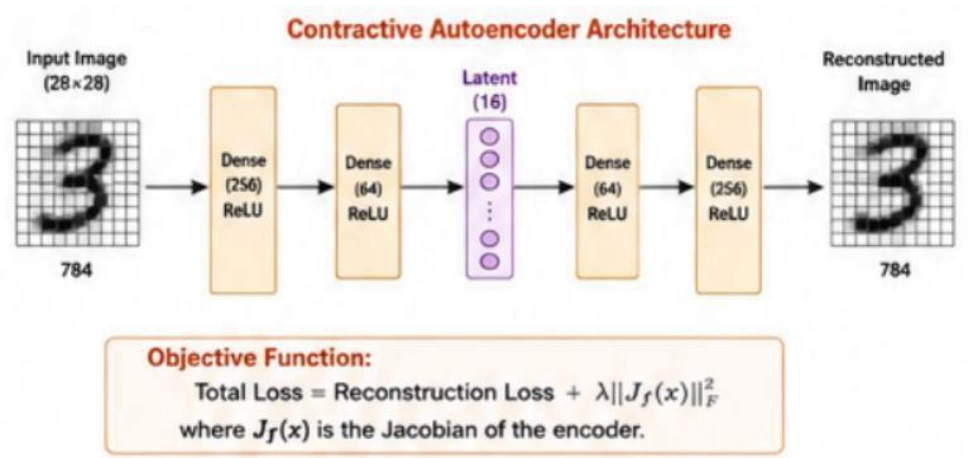
Proposed model: Contractive Autoencoder

Architecture:

Input Image 784 → Dense(256) → Dense(64) → Latent(16) → Dense(64) → Dense(256) → Output Image 784

Design:

- Input: 28 × 28 grayscale image = 784 features.
- Encoder: 784 → 256 → 64 → 16.
- Latent representation: 16 features.
- Decoder: 16 → 64 → 256 → 784.
- Hidden activation: **ReLU**.
- Output activation: **Sigmoid** for normalized image pixels.



Objective function:

Total Loss = Reconstruction Loss + λ‖Jf(x)‖²F

where **Jf(x)** is the encoder Jacobian.

Robustness mechanism: The contractive penalty minimizes the sensitivity of the latent representation to small changes in the input.

Working: If an image has small pixel-level changes, the encoder produces a similar latent representation, making the learned features stable.

Evaluation: Reconstruction MSE, classification accuracy under noise, robustness to perturbations and latent-feature stability.

Expected outcome: The model learns stable image features and becomes less sensitive to small variations and noise.

5. Design a GAN-Based Data Generation System

Real-world problem: An organization has limited training images and wants realistic synthetic images.

Requirements:

- **Input:** Random noise vector.
- **Output:** Realistic synthetic images.
- **Constraint:** Generated images should be realistic and diverse.

Architecture:

Random Noise z → Generator → Synthetic Image → Discriminator → Real/Fake

Generator

- Input: 100-dimensional random noise vector.
- Dense/upsampling layers.
- ReLU activation in hidden layers.
- Tanh activation at output.
- Output: Synthetic image.

Discriminator

- Input: Real or generated image.
- Convolutional layers.
- Leaky ReLU activation.
- Sigmoid output.
- Output: Probability of **Real or Fake**.

Adversarial Training:

1. Generator creates a synthetic image from random noise.
2. Discriminator receives both real and generated images.
3. Discriminator learns to distinguish real from fake.

4. Generator learns to fool the discriminator.
5. Both networks improve through adversarial training.

Loss functions:

- **Discriminator:** Binary Cross-Entropy loss.
- **Generator:** Adversarial Binary Cross-Entropy loss.

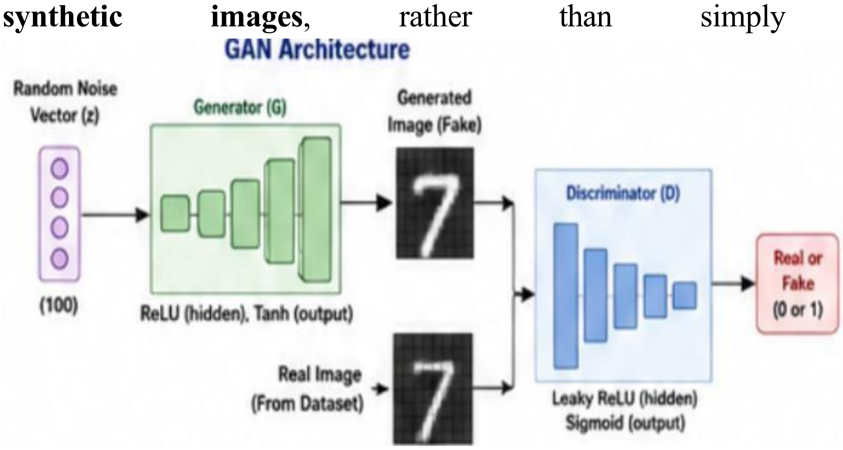
Evaluation:

- Image quality.
- Diversity of generated images.
- FID (Fréchet Inception Distance).
- Precision and recall for generated distributions.
- Human visual evaluation.

GAN vs Autoencoder

Feature	**Autoencoder**	**GAN**
Main purpose	Reconstruction	Data generation
Training	Reconstruction loss	Adversarial training
Output quality	Usually good reconstruction	Often highly realistic
Generation	Limited	Strong
Main components	Encoder + Decoder	Generator + Discriminator

Justification: A GAN is more suitable because the main requirement is to generate **realistic and diverse synthetic images**, rather than simply reconstruct existing image



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 Exemplary	Level 3 Proficient	Level 2 Developing	Level 1 Beginning
Understanding of Problem Context	20	Clearly understands the problem, requirements, constraints, and expected outcomes.	Good understanding with minor gaps in requirements.	Basic understanding; some requirements are unclear.	Poor understanding or misinterpretation of the problem.
Application of Concepts	30	Accurately applies appropriate generative deep learning concepts and architectures.	Applies relevant concepts correctly with minor errors.	Applies basic concepts with limited accuracy.	Fails to apply appropriate concepts.
Design Approach	20	Innovative, logical, well-structured, feasible, and technically justified design.	Logical and feasible design with minor gaps.	Basic design with some inconsistencies.	Unclear, illogical, or impractical design.
Accuracy of Solution	20	Completely correct architecture, process, objectives, and justification.	Mostly correct with minor technical errors.	Partially correct solution with noticeable gaps.	Incorrect or incomplete solution.
Clarity	10	Clear, well-organized diagram and explanation with proper technical labeling.	Understandable with minor clarity issues.	Limited clarity and explanation.	Poorly presented and difficult to understand.